



Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Kontrol Yapıları ve Fonksiyonlar

PyGame ile Oyun Geliştirme – Hafta 2

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

- 1 Koşullu İfadeler
- 2 Döngüler
- 3 Fonksiyonlar
- 4 Veri Yapıları Temelleri
- 5 Mini Proje ve Özet

Koşullu İfadeler

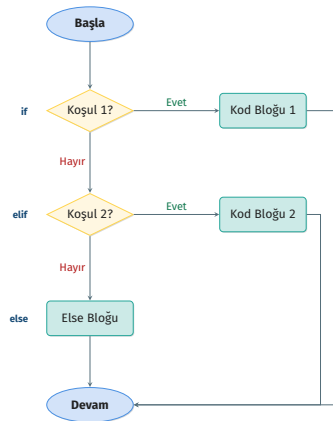
if İfadesi

```
oyuncu_can = 100

if oyuncu_can > 0:
    print("Oyuncu hayatta!")
```

Önemli Noktalar

- Koşuldan sonra :
- Girinti (4 boşluk) zorunlu
- Koşul True ise kod çalışır
- = atama, == karşılaştırma



if-elif-else: Çoklu Seçenekler

```
can = 35

if can >= 70:
    print("Sağlık: İyi")
elif can >= 30:
    print("Sağlık: Orta")
elif can >= 10:
    print("Sağlık: Düşük")
else:
    print("Sağlık: Kritik!")
```

Dikkat

Koşullar yukarıdan aşağıya kontrol edilir.
İlk doğru koşul bulununca diğerleri atlanır!

Kurallar

- Bir if (zorunlu)
- Sınırsız elif
- En fazla bir else

```
if oyuncu_hayatta:
    if dusman_gorunur:
        if menzilde:
            if mermi > 0:
                print("Ates!")
            else:
                print("Mermi yok")
        else:
            print("Yaklasıyorum")
```

Alternatif: and ile

```
if (oyuncu_hayatta and
    dusman_gorunur and
    menzilde and
    mermi > 0):
    print("Ates!")
```

İpucu

Derin iç içe koşullar yerine mantıksal operatörler tercih edilebilir.

Koşullarda Mantıksal Operatörler

and – Tüm koşullar doğru:

```
if can > 0 and mermi > 0:  
    print("Ates et!")
```

in – Üyelik testi:

```
if "kılıç" in envanter:  
    print("Silahlili!")
```

or – En az biri doğru:

```
if anahtar or maymuncuk:  
    print("Kapi açıldı!")
```

not – Tersine çevirme:

```
if not oyun_bitti:  
    print("Devam!")
```

Karmaşık Koşul

```
# Buyu yapabilir mi?  
if (mana >= 30 and  
    seviye >= 5 and  
    not susturulmus):  
    print("Buyu!")
```

Döngüler

while Döngüsü

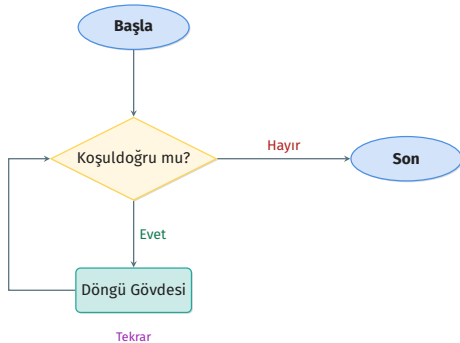
```
sayac = 5

while sayac > 0:
    print(f"Geri sayim: {sayac}")
    sayac -= 1

print("Kalkis!")
```

Oyun döngüsü:

```
while oyun_aktif:
    olaylari_isle()
    guncelle()
    ciz()
```



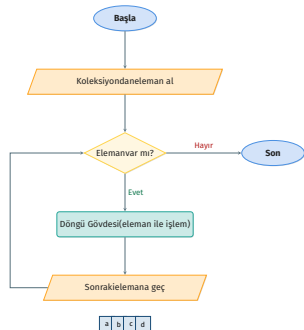
for Döngüsü

```
envanter = ["kilic", "kalkan",  
            "iksir"]
```

```
for esya in envanter:  
    print(f"- {esya}")
```

enumerate() ile indeks:

```
for i, esya in enumerate(envanter):  
    print(f"{i+1}. {esya}")
```



range() Fonksiyonu

```
for i in range(5):           # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):       # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)

for i in range(5, 0, -1):   # 5, 4, 3, 2, 1
    print(f"Geri sayim: {i}")
```

Kullanım

range(n) range(başla, bit) range(başla, bit, adım)

break ve continue

break – Döngüyü sonlandır:

```
for dusman in dusmanlar:
    if dusman == aranan:
        print("Bulundu!")
        break
```

continue – İterasyonu atla:

```
for dusman in dusmanlar:
    if dusman["can"] <= 0:
        continue # Oluyu atla
    print(dusman["isim"])
```

while True kalıbı:

```
while True:
    komut = input("> ")
    if komut == "cikis":
        break
    print(f"Komut: {komut}")
```

while vs for

while: Koşula bağlı tekrar
for: Koleksiyon gezme veya
belli sayıda tekrar

Fonksiyonlar

Fonksiyon Tanımlama

```
def selamla(isim):  
    print(f"Merhaba, {isim}!")  
  
selamla("Ahmet")      # Merhaba, Ahmet!  
selamla("Zeynep")     # Merhaba, Zeynep!
```

Neden Fonksiyon?

- Tekrar kullanım
- Kod organizasyonu
- Okunabilirlik
- Bakım kolaylığı

İsimlendirme

- snake_case kullan
- Fiil ile başla
- hasar_hesapla()
- oyunu_baslat()
- menu_goster()

Parametreler ve Argümanlar

```
# Birden fazla parametre
def karakter_olustur(isim, sinif, seviye=1):
    print(f"{isim} ({sinif}) - Sv.{seviye}")

karakter_olustur("Ali", "Savasci")
karakter_olustur("Veli", "Buyucu", seviye=5)
```

Tür	Örnek
Konumsal	saldir("kilic", 20)
Anahtar kelime	saldir(silah="kilic", hasar=20)
Varsayılan değer	def saldir(silah="yumruk"):

return İfadesi

```
def hasar_hesapla(guc,
                  silah_bonus,
                  zirh):
    ham = guc + silah_bonus
    gercek = ham - zirh
    return max(0, gercek)
```

Kullanım

```
h = hasar_hesapla(20, 10, 15)
print(f"Hasar: {h}") # 15
```

```
# Boolean dondurme
def hayatta_mi(can):
    return can > 0

# Birden fazla deger
def konum_al():
    return 100, 200

x, y = konum_al()
```

Dikkat

return değer döndürür,
print ekrana yazar.
Farklı şeylerdir!

Değişken Kapsamı (Scope)

```
puan = 0  # Global degisken

def puan_ekle(miktar):
    # puan burada okunamaz!
    # global kullanmak gerekir
    global puan
    puan += miktar

def puan_goster():
    # Okumak icin global gerekmez
    print(f"Puan: {puan}")

puan_ekle(50)
puan_goster()  # Puan: 50
```

Kapsam Kuralları

- **Yerel:** Fonksiyon içinde tanımlı, dışarıdan erişilemez
- **Global:** Her yerden erişilebilir
- `global` anahtar kelimesi ile değiştirilebilir

İyi Uygulama

`global` yerine parametre ve return tercih edin.

Veri Yapıları Temelleri

Listeler

```
envanter = ["kilic", "kalkan", "iksir"]

# Erisim
print(envanter[0])      # kilic
print(envanter[-1])     # iksir

# Ekleme / Silme
envanter.append("anahtar")
envanter.remove("iksir")

# Uzunluk ve üyelik
print(len(envanter))    # 3
print("kilic" in envanter) # True
```

Yararlı Metotlar

append() insert() remove() pop() sort() reverse()

```
oyuncu = {  
    "isim": "Kahraman",  
    "can": 100,  
    "guc": 25,  
    "envanter": ["kilic"]  
}  
  
# Erisim  
print(oyuncu["can"])           # 100  
print(oyuncu.get("mana", 0))   # 0 (guvenli)  
  
# Guncelleme  
oyuncu["can"] = 80  
oyuncu["mana"] = 50
```

Ne Zaman Sözlük?

Anahtar-değer çiftleri gerektiğinde: oyuncu özellikleri, düşman istatistikleri, ayarlar.

Demetler (Tuple)

```
# Degistirilemez koleksiyon
boyut = (800, 600)
kirmizi = (255, 0, 0)

# Acma (unpacking)
x, y = boyut
print(f"x={x}, y={y}")

# PyGame'de cok kullanilir
ekran = (800, 600)
renk = (0, 128, 255)
konum = (100, 200)
```

	Liste	Demet
Yazım	[]	()
Değişir	Evet	Hayır
Hız	Yavaş	Hızlı
Kullanım	Envanter	Koordinat

İpucu

Değişmeyecek veriler için demet, değişecekler için liste kullanın.

Mini Proje ve Özet

Mini Proje: Metin Tabanlı Macera

Özellikler

- Zindan keşfi (10 oda)
- Düşmanlarla savaş
- Hazine toplama
- Envanter sistemi

Kullanılan Kavramlar

- if/elif/else
- while/for
- Fonksiyonlar
- Liste ve sözlükler

```
oyuncu = {  
    "can": 100,  
    "guc": 15,  
    "altin": 0,  
    "envanter": []  
}  
  
def savas(dusman_adi):  
    # Savas mekanigi  
    ...  
  
while (mevcut_oda < 10 and  
       oyuncu["can"] > 0):  
    # Rastgele oda olaylari  
    olay = random.choice(  
        ["bos", "hazine",  
         "tuzak", "dusman"])  
    ...
```

Koşullu İfadeler

- if, elif, else
- and, or, not, in
- İç içe koşullar

Döngüler

- while: Koşula bağlı tekrar
- for: Koleksiyon gezme
- range(), break, continue

Fonksiyonlar

- def ile tanımlama
- Parametre ve varsayılan değer
- return ile değer dönme
- Yerel ve global kapsam

Veri Yapıları

- Liste: Sıralı, değişebilir
- Sözlük: Anahtar-değer
- Demet: Değişmez

Gelecek Hafta: PyGame'e giriş! Oyun penceresi ve ilk grafikler.

- Kitaptan Bölüm 2'yi okuyun
- Lab egzersizleri:
 - 1 Çarpım tablosu programı yazın (for ile)
 - 2 FizzBuzz problemi çözün
 - 3 Asal sayı kontrol fonksiyonu yazın
 - 4 Metin tabanlı macera oyununu çalıştırın ve geliştirin:
 - Dükkan sistemi ekleyin
 - Farklı silahlar ekleyin
 - Karakter sınıfları (Savaşçı, Büyücü) ekleyin

İletişim

tberber@ktu.edu.tr

Sorular?

Teşekkürler!